Deliverable 1
Sparse Matrix-Vector Multiplication (SpMV) Investigation
GPU Computing 2025-2026

Matilde Munaretto
GitHub repository: https://github.com/matilde03-research/GPU_Computing
Master's Degree in Artificial Intelligence Systems , DISI, UniTN
matilde.munaretto@studenti.unitn.it matricola: 266680

***Abstract*—Sparse Matrix-Vector Multiplication (SpMV) is a fundamental kernel in scientific computing, machine learning, and graph analytics, yet remains difficult to optimize due to its memory-bound nature and the irregular sparsity patterns of real-world matrices. This work investigates the performance of SpMV across mainly three storage formats (COO, CSR and ELL) on an NVIDIA A30 GPU using CUDA, evaluated on 10 diverse sparse matrices from the SuiteSparse collection. Multiple algorithmic variants are implemented and analyzed, including warp-level shuffle reductions, segmented scans, coalesced memory access patterns, and flat workgroup decomposition. Results reveal that no single format or kernel consistently dominates across all matrices; performance is strongly shaped by the interplay between matrix sparsity structure, row-length distribution, and kernel design. cuSPARSE provides the most reliable baseline, while custom kernels can outperform it on specific matrix types.**

***Index Terms*—SpMV, GPU Computing, kernels, memory-bound, warp, threads, coalescing, shared memory**

I. Introduction

Sparse Matrix-Vector Multiplication (SpMV) is a fundamental computational kernel widely used in scientific computing, engineering simulations, machine learning, graph analytics, and data mining applications. Many iterative numerical methods and sparse linear algebra algorithms repeatedly rely on SpMV operations, making its performance a critical factor in overall application efficiency. Formally, given a sparse matrix $A \in \mathbb{R}^{m \times n}$ and dense vectors $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$, the SpMV operation is defined as

$$y = Ax \tag{1}$$

Despite its simple mathematical formulation, achieving high performance for SpMV remains challenging [1]. Unlike dense matrix operations, sparse matrices store and process only non-zero elements, leading to irregular memory access patterns and reduced data locality. The distribution of non-zero elements can vary significantly across matrices arising from different applications, creating load imbalance and limiting opportunities for regular parallel execution.

A major characteristic of SpMV is that it is predominantly a memory-bound computation problem rather than a compute-bound one. The arithmetic intensity of the operation is inherently low: each non-zero element typically contributes only a multiply-add operation while requiring multiple memory accesses for matrix values, column indices, and vector elements. As a result, performance is largely constrained by memory bandwidth and data movement rather than by the computational capabilities of the processor.

For this reason, optimizing SpMV on GPU architectures has attracted extensive attention over several decades. The combination of irregular sparsity patterns, diverse sparse matrix storage formats, and increasingly heterogeneous hardware architectures creates a complex optimization landscape. Numerous approaches have been proposed to improve memory access efficiency, increase parallelism, and reduce load imbalance. However, due to the wide variety of sparse matrices and hardware characteristics, no single optimization strategy consistently achieves optimal performance across all scenarios.

Methodology

Our experimental methodology evaluates the performance of sparse matrix-vector multiplication (SpMV) across multiple storage formats and computational implementations. The main objective of the different sparse matrix storage formats is to remove all the zero elements from the matrix representation. Removing all the zero elements not only saves storage but also eliminates the need to fetch these zero elements from memory and perform useless multiplication or addition operations with them [5]. Four distinct matrix storage formats have been examined: Coordinate (COO), Compressed Sparse Row (CSR), Ellpack (ELL), and the cuSPARSE library implementation, which provides highly optimized vendor-supplied kernels. For CPU implementations it was developed a single-threaded baseline kernels written in C++ with OpenMP parallelization, which serve as the reference for result validation. GPU implementations were developed in CUDA utilizing architecture-specific optimizations, including warp-level shuffle reductions,

coalesced memory access patterns, and cached vector loads via `__ldg()` intrinsics, compiled with NVCC targeting compute capability SM_75 (`-arch=sm_75` flag) with C++11 standard (`-std=c++11`). Validation was performed by comparing GPU-computed results against CPU baselines using both absolute (tolerance: 1×10^{-3}) and relative error metrics (tolerance: 1×10^{-4}), reporting first five discrepancies and maximum error magnitude for each kernel variant. Performance measurement methodology included warmup cycles followed by timed iterations using `std::chrono` high-resolution timers, capturing kernel execution time and computing throughput in GFLOPS. Experiments were conducted on an NVIDIA A30 GPU (with CUDA 11.8.0) accessed through batch job scheduling, processing real-world sparse matrices in Matrix Market format, executing repeated iterations to establish statistical significance of timing measurements. The experiments were run with 4 warmup cycles and averaging the results over 100 iterations per kernel.

II. Dataset

The dataset is composed of 10 sparse matrices downloaded from SuiteSparse [4] and also taken into consideration by Chu et al. (2023). A detailed description of the dataset used is given in the Table I. As regards the implementation of the random dense vector to be multiplied with the sparse matrix, a random vector with a size compatible with that of the matrix was created with seed set equal to 42 to allow the reproducibility of the experiments and compliant with the Float32 data type.

III. Results

In this section results are shown in Table II and Table III and further discussed in the following section.

IV. Discussion

In this section we will analyze each one of the kernels' implementations to understand better the results we obtained:

- COO-Standard: each thread processes one non-zero element independently, using atomicAdd to accumulate results into the output vector. It is simple to implement, format-agnostic (no preprocessing required) and works on unordered data. Its disadvantages are severe atomic contention when multiple threads target the same row and throughput degrades with dense rows and irregular sparsity patterns.
- COO-SortedSegmentedReduction: data is sorted by row index on the host, then threads use warp-level shuffle operations and segmented scans to reduce atomicAdd calls by accumulating values within warps before flushing to output. Reduces atomic contention through intra-warp aggregation, there is a minimal preprocessing overhead (host-side sort) and it performs a better load balancing across warps. Its disadvantages are that it requires presorting (one-time host cost, not included in timing), that it still uses atomicAdd but with fewer calls and memory access patterns are less coalesced than CSR variants.

Algorithm 1 COO SpMV — Sorted Segmented Reduction

Require: `nnz`, arrays `row_idx`, `col_idx`, `values`, `x`, output `y`

tid ← `blockIdx.x` × `blockDim.x` + `threadIdx.x`
lane ← `threadIdx.x` mod `WARP_SIZE`
val ← 0, *row* ← −1
if *tid* < `nnz` **then**
  row ← `row_idx`[*tid*]
  val ← `values`[*tid*] × `x`[`col_idx`[*tid*]]
end if
▷ Warp-local segmented inclusive scan
for *offset* ← 1 **to** `WARP_SIZE`/2 **step** ×2 **do**
  nb_val ← `__shfl_up`(*val*, *offset*)
  nb_row ← `__shfl_up`(*row*, *offset*)
  if *lane* ≥ *offset* **and** *nb_row* = *row* **then**
    val ← *val* + *nb_val*
  end if
end for

if *tid* ≥ `nnz` **then return**
end if
▷ Determine whether this thread ends a segment
next_row ← `__shfl_down`(*row*, 1)
end_segment ← (*lane* = `WARP_SIZE` − 1) **or** (*tid* = `nnz` − 1) **or** (*next_row* ≠ *row*)
if *end_segment* **then**
  `atomicAdd`(`y`[*row*], *val*)
end if

- CSR-Vector [3]: each warp (32 threads) is assigned one row, threads within the warp cooperatively reduce the dot product using stride-based loading and warp shuffle reduction. Excellent load balancing for matrices with moderate row length, there are no atomic operations (implicit row assignment), there is coalesced memory access for column indices and values and efficient warp-level reduction with shuffle instructions. Its disadvantages are that it underutilizes resources on matrices with many very short rows ($<$ 32 NNZ) and it incurs divergence and idle threads when rows have fewer non-zeros than warp size.
- CSR-Flat [2]: load balancing is decoupled from row structure, the shared memory caching improves L1 hit rates for repeated column accesses, there is an effective use of CUDA block resources and the kernel scales well on wide matrices. Its disadvantages are that it requires a preprocessing phase, atomicAdd is still present for rows spanning multiple workgroups and shared memory footprint is large (STRIDE_FLAT entries).
- CSR-Line-Enhance [2]: there are no atomic operations to output vector (major advantage), it adapts configuration based on matrix structure (V and N_ROWS chosen per avg NNZ/row), it eliminates atomic contention entirely and it is very efficient for structured matrices. Its disadvantages are that it has a more complex implementation with template instantiation, it requires careful synchronization and shared memory management, and it may happen to have branch divergence if workgroups don't align perfectly with row boundaries.
- ELL-Basic: one thread per row, for each row, a single

TABLE I
SPARSE MATRIX DATASETS USED IN THE EVALUATION

Matrix	**Rows**	**Cols**	**NNZ**	**Description**
ASIC_680ks	682,712	682,712	2,329,176	Circuit simulation problem. Highly irregular sparsity pattern. Row lengths vary considerably, and nonzeros tend to cluster.
bone010	986,703	986,703	36,326,514	Model reduction problem. Relatively high density. Row degrees are uneven, some rows denser than others.
boyd2	466,316	466,316	890,091	Optimization problem. Exhibiting block-like structures and moderate row-length variation.
eu-2005	862,664	862,664	19,235,140	Directed graph matrix. Power-law sparsity structure. Characterized by graph-community patterns.
FullChip	2,987,012	2,987,012	26,621,990	Circuit simulation matrix. Extreme sparsity and strong structural irregularity. Row densities vary significantly.
Ga41As41H72	268,096	268,096	9,378,286	Theoretical quantum chemistry matrix. Nonzeros appear in clustered bands and localized interaction regions.
ldoor	952,203	952,203	23,737,339	Matrix with a relatively regular sparsity pattern. Nonzeros are concentrated around neighboring elements.
rajat31	4,690,002	4,690,002	20,316,253	Circuit simulation matrix. Low average row density and highly uneven distribution. The sparsity pattern lacks regularity and exhibits fill regions around connected components.
Rucci1	1,977,885	109,900	7,791,168	Least-squares problem with a tall-and-skinny structure.
webbase-1M	1,000,005	1,000,005	3,105,536	Weighted directed graph with highly skewed degree distribution. Most rows are short while a small number are extremely long.

TABLE II
PERFORMANCE COMPARISON OF COO AND CSR SPARSE MATRIX IMPLEMENTATIONS

Matrix	**COO-Standard**		**COO-SortedSegmentedReduction**		**CSR-Vector**		**CSR-Flat**		**CSR-Line-Enhance**	
	Time (ms)	*GFLOP/s*	*Time (ms)*	*GFLOP/s*	*Time (ms)*	*GFLOP/s*	*Time (ms)*	*GFLOP/s*	*Time (ms)*	*GFLOP/s*
ASIC_680ks	0,0963	48,37	0,0624	74,7	1141,0213	8,97	1140,6592	59,51	1140,6271	37,25
bone010	4,5108	31,78	1,2492	114,74	47321,0543	103,39	47321,0702	105,61	47320,5352	165,27
boyd2	0,7488	4,01	0,0599	50,08	829,6701	5,25	829,1740	64,7	8308,0200	1,76
eu-2005	0,7043	54,62	0,4360	88,24	12880,9177	49,63	12880,6292	85,92	12880,5094	104,88
FullChip	7,6789	6,93	0,7631	69,77	18080,0885	5,48	18071,0756	83,28	18097,6793	1,95
Ga41As41H72	1,7323	21,35	0,3690	100,21	12176,9798	108,07	12177,1064	84,22	12176,9726	110,43
ldoor	3,0284	30,72	0,8086	115,06	31263,7002	87,98	31263,6237	99,73	31263,2505	153,05
rajat31	0,8669	46,87	0,4649	87,4	11098,8741	13,82	11096,4791	87,63	11096,4611	77,17
Rucci1	0,2616	59,57	0,1811	86,04	5330,9887	10,9	5329,7755	91,74	5329,7680	74,4
webbase-1M	0,1398	44,42	0,0940	66,07	1864,4945	8,19	1863,8781	62,49	1864,5891	7,28

TABLE III
PERFORMANCE COMPARISON OF ELL IMPLEMENTATIONS AND CUSPARSE ("OOM" STANDS FOR AN OUT OF MEMORY CASE

Matrix	**ELL-Basic**		**ELL-Coalesced**		**cuSPARSE**	
	Time (ms)	*GFLOP/s*	*Time (ms)*	*GFLOP/s*	*Time (ms)*	*GFLOP/s*
ASIC_680ks	2494,8448	5,77	2494,1261	52,35	0,0606	76,84
bone010	0,8328	172,11	0,1352	1060,24	0,9573	149,72
boyd2	oom	oom	oom	oom	0,0387	77,57
eu-2005	oom	oom	oom	oom	0,2757	139,54
FullChip	oom	oom	oom	oom	0,4622	115,19
Ga41As41H72	1,1548	32,02	0,1658	223,01	0,2654	139,32
ldoor	0,7171	129,76	0,1260	738,67	0,5714	162,82
rajat31	oom	oom	oom	oom	0,4057	100,15
Rucci1	0,0975	159,76	0,0318	490,72	0,1596	97,63
webbase-1M	oom	oom	oom	oom	0,0650	95,63

thread reads max_row_len entries from the ELL array (column-major storage where entry (row, j) is at index j·m + row), skips padded entries (marked with col_idx=-1), and accumulates the dot product using cached reads (__ldg). It has a simple kernel logic, it uses L1 cache effectively via __ldg() for x-vector reads, there is no synchronization overhead or shared memory contention. Its disadvantage is severe memory inefficiency on irregular matrices: ELL requires padding to max_row_len, wasting memory proportional to row-length variance. On matrices with extreme variance (e.g., boyd2, max_row_len=8308 vs avg 1.76 NNZ/row), this causes out-of-memory failures. Other disadvantages are thread underutilization when rows are short and no inter-thread cooperation

- ELL-Coalesced: each warp (32 threads) cooperatively processes one row, the threads are strided by warp size (lane 0 reads entry 0, lane 1 reads entry 32, etc.) to improve cache locality. Furthermore Warp shuffle reduction aggregates partial sums and only lane 0 writes the result. Better cache locality and memory coalescing than Basic, warp-level parallelism increases throughput per row compared to single-thread processing, shuffle reduction is very efficient. Its disadvantages are: inherits ELL's fundamental memory inefficiency, the padding waste is identical to Basic, out of memory bound failures on highly irregular matrices persist. Warp underutilization is even worse when rows have < 32 NNZ.

Algorithm 2 ELL SpMV — Coalesced (warp-per-row)

Require: m, `max_row_len`, arrays `col_idx`, `values`, `x`, output `y`

warp_id ← (`blockIdx.x` × `blockDim.x` + `threadIdx.x`) / 32
lane ← `threadIdx.x` mod 32
if *warp_id* $\geq m$ **then return**
end if
row ← *warp_id*, *sum* ← 0
▷ Each lane processes entries strided by warp size
for $j \leftarrow$ *lane* **to** `max_row_len` $- 1$ **step** 32 **do**
  idx ← $j \times m +$ *row*
  col ← `col_idx`[*idx*]
  if *col* $\neq -1$ **then**
    sum ← *sum* + `values`[*idx*] × `x`[*col*]
  end if
end for
▷ Warp-level reduction via shuffle down
for *offset* ← 16 **downto** 1 **step** ÷2 **do**
  sum ← *sum* + `__shfl_down`(*sum*, *offset*)
end for
▷ Lane 0 writes the result
if *lane* $= 0$ **then**
  `y`[*row*] ← *sum*
end if

- cuSPARSE (NVIDIA Reference Implementation): Highest consistency and reliability across all matrix types. This kernel was just used as a safe reference.

V. Conclusions

This study offers several practical lessons regarding the design and selection of formats and SpMV kernels for GPU execution. The most significant finding is that performance is inherently matrix-dependent: no kernel universally dominates across all tested matrices. COO-SortedSegmentedReduction consistently delivers competitive throughput (up to 115 GFLOP/s) through intra-warp aggregation, reducing atomic contention compared to the standard COO kernel. CSR-based kernels benefit greatly from coalesced memory access and warp-level reductions, with CSR-Line-Enhance achieving the highest throughput on several structured matrices (e.g., 165 GFLOP/s on bone010). ELL-Coalesced, when memory allows, achieves the highest peak throughput observed (over 1060 GFLOP/s on bone010) by exploiting warp-level parallelism and cache locality. cuSPARSE, while not always the fastest, is the only kernel that provides a strong, reliable reference, and it's also quite fast. Regarding the limitations the ELL format's is clearly the most limited kernel. The fundamental limitation is the need to pad all rows to the maximum row length (results in catastrophic memory waste and out-of-memory failures on highly irregular matrices such as boyd2, eu-2005, FullChip, rajat31, and webbase-1M). CSR-Vector suffers from severe thread underutilization on matrices with many short rows, as seen by the anomalously high execution times recorded for several matrices. COO-Standard is limited by atomic contention, making it unsuitable for matrices with dense rows or skewed non-zero distributions. Timing measurements do not include preprocessing costs (e.g., host-side sorting for COO-Sorted or ELL array construction), which could impact real-world end-to-end performance. When the sparsity pattern is known in advance and rows are relatively uniform, ELL-Coalesced offers exceptional throughput. For irregular matrices with highly variable row lengths, CSR-based variants (particularly CSR-Flat or CSR-Line-Enhance) are safer choices that avoid memory failures while still achieving good bandwidth utilization. COO-SortedSegmentedReduction is a practical middle ground that requires no format conversion and performs well across diverse matrix types. When portability and correctness are priorities, cuSPARSE is the recommended choice, as it handles all matrix structures reliably and fast. Ultimately, an adaptive selection strategy (choosing the kernel based on lightweight matrix statistics such as average and maximum NNZ per row) is likely the most effective approach for production systems.

References

[1] Gao, J., Liu, B., Ji, W. and Huang, H., 2024. A systematic literature survey of sparse matrix-vector multiplication. arXiv preprint arXiv:2404.06047.

[2] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient Algorithm Design of Optimizing SpMV on GPU. In Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '23), 2023. DOI: 10.1145/3588195.3593002.

[3] Bell, N., and Garland, M. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In SC '09, 2009.

[4] University of Florida, *SuiteSparse Matrix Collection*, Month Year. [Online]. Available: https://sparse.tamu.edu/. [Accessed: 04/05/2026].

[5] Wen Mei et al, *Programming Massively Parallel Processor*, fourth edition.